

MODELLING ILLUSTRATIONS USING GAN

A THESIS REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS

FOR THE AWARD OF THE DEGREE
OF
BACHELOR OF TECHNOLOGY
IN
DIVISION OF COMPUTER ENGINEERING

Submitted by:

Vaibhav Sharan (2017UCO1677)
S.M. Aadithya (2017UIT2520)
MD MEHRAN (2017UCO1651)

Under the Supervision of
Dr. Ankita Bansal
Dr. Kushil Saini



DIVISION OF COMPUTER ENGINEERING
NETAJI SUBHAS INSTITUTE OF TECHNOLOGY
UNIVERSITY OF DELHI
JUNE, 2021

ACKNOWLEDGEMENT

It is near impossible to work on a project without the encouragement & assistance of other people. Our project is no exception to this. We would like to express our sincere gratitude to our supervisors Prof. Ankita Bansal and Prof. Kushil Saini for providing their invaluable guidance, comments and suggestions throughout the course of the project. They were the ones who constantly motivated and encouraged us to work harder. We thank NSIT for providing us with such a good opportunity. We thank our parents, friends and family members who have always supported us. We would also give our thanks to our colleagues in directly or indirectly helping us develop the project and people who have willingly helped us out with their abilities. The project was an extremely productive & enriching experience, not only technically but also in the matter of providing some practical skills.



Department of Computer Engineering

University of Delhi

Delhi-110007, India

CERTIFICATE OF ORIGINALITY

We, Vaibhav Sharan (2017UCO1677) of B.Tech Department of Computer Engineering, S.M. Aadithya (2017UIT2520), Md Mehran (2017UCO1651) of B.Tech, Department of Information Technology. Hereby declare that the Project-Thesis titled “Modelling Illustrations using GAN” which is submitted by us to the Department of Computer Engineering, Netaji Subhas Institute of Technology, Delhi (University of Delhi) in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology, is original and not copied from source without proper citation. The manuscript has been subjected to plagiarism check by Turnitin software. This work has not previously formed the basis for the award of any Degree.

Place: Delhi

Date: 28/06/21

Vaibhav Sharan

S.M. Aadithya

Md Mehran



Department of Computer Engineering

Netaji Subhas University of Technology

Delhi-11007, India

CERTIFICATE OF DECLARATION

This is to certify that the work embodied in the Project-thesis titled “Modelling Illustrations using GAN” has been completed by S.M. Aadithya (2017UIT2520), Md Mehran (2017UCO1651) of B.Tech, Department of Computer Science, and Vaibhav Sharan (2017UCO1677) of B.Tech, Department of Computer Science, under the guidance of Dr. Ankita Bansal and Dr. Kushil Saini towards fulfillment of the requirements for the award of the degree of Bachelor of Technology. This work is based on original research and has not been submitted in full or in part of any other diploma or degree of any university.

Place: Delhi

Date: 28/06/21

Dr. Ankita Bansal

Dr. Kushil Saini

ABSTRACT

Automatic generation of facial images has been well studied after the Generative Adversarial Network (GAN) became a well-known framework. Our goal is to explore the training of DCGAN models specialized on an animated facial image dataset. Its purpose is to automatically generate faces of animated characters based on the facial features of the input training data.

It involves tasks like importing the image through image acquisition tools, analysing and manipulating the image to get output in which result can be altered image or even a report that is based on image analysis.

Despite the breakthroughs in accuracy and speed of single image super-resolution using faster and deeper CNNs, Recent work has largely focused on minimizing the mean squared reconstruction error. The resulting estimates have high peak signal-to-noise ratios, but they are often lacking high-frequency details and are perceptually unsatisfying in the sense that they fail to match the fidelity expected at the higher resolution.

Our aim is to create a model which, given a set of input tags, will output a set of generated images corresponding to the tags. We use both GAN components (generator and discriminator) which will compete and thus with training will return a model which will be capable enough where the generator can generate upsampled, good quality images from low quality images.

INDEX

ACKNOWLEDGEMENT	2
CERTIFICATE	3
DECLARATION	4
ABSTRACT	5
INDEX	6
LIST OF FIGURES	8
CHAPTER 1	9-10
INTRODUCTION	
1.1 Thesis Introduction	
1.2 Motivation	
CHAPTER 2	11-18
RELATED WORK	
2.1 Generative Adversarial Network	
2.2 Basic Architecture	
2.2.1 Adversarial Network Architecture	
2.3 Components of GAN	
2.3.1 Convolutional Neural Networks	
2.3.2 Challenges to a CNN	
2.3.3 Loss Functions	
2.3.4 Activation Function	

2.3.5 Backpropagation

CHAPTER 3 19-24

METHODOLOGY

3.1 Our Approach

3.2 Design of the model

3.3 Data set and Experimental Tools setup

CHAPTER 4 25-27

RESULTS AND APPLICATIONS

4.1 Results

4.2 Applications

4.3 Qualitative Analysis

REFERENCES 28

LIST OF FIGURES:

Fig 1. A generated facial image

Fig 2. GAN basic principle

Fig 3. Generator Model Architecture

Fig 4. Discriminator Model Architecture

Fig 5. CNN Components

Fig 6. Activation Functions

Fig 7. Our GAN's approach

Fig 8. Images during training (1 picture per 20 epochs)

Fig 9. Sample training images.

Fig 10. Results after 5000 epochs

CHAPTER 1

INTRODUCTION

1.1 Thesis Introduction

Automatic generation of facial images has been well studied after the Generative Adversarial Network (GAN) became a well-known framework. Our goal is to explore the training of DCGAN models specialized on an animated facial image dataset. Its purpose is to automatically generate faces of animated characters based on the inputted facial feature tags.

The project consists of four main steps:

1. Review and study recent works on the topic.
2. Implement one or more of the solutions and possibly compare pre-trained versions.
3. Get some data, train and test the model.
4. Improve the model and the data to get better results.

1.2 Motivation

Automatic generation of facial images has been well studied after the Generative Adversarial Network(GAN) came out. There exists some attempts applying the GAN model to the problem of generating facial images of anime characters, but none of the existing work gives a promising result. In

this work, we explore the training of GAN models specialized on an anime facial image dataset. We address the issue from both the data and the model aspect, by collecting a more clean, well-suited dataset and leverage proper, empirical application of DCGAN.

With qualitative and quantitative analysis and case studies we demonstrate that our efforts lead to a stable and high-quality model.



Fig 1. A generated facial image.

CHAPTER 2

RELATED WORK

2.1 Generative Adversarial Network

GANs are a class of AI algorithms based on Unsupervised Machine Learning. GANs are deep neural network architectures comprised of two networks (Generator and Discriminator) competing with each other (thus the “adversarial”). GANs is about creating, like drawing a portrait or composing a symphony. The main focus for GANs is to generate data from scratch.

To understand GANs, first we need to understand what a generative model is. In machine learning, the two main classes of models are generative and discriminative. A discriminative model is one that discriminates between two (or more) different classes of data, for example a convolutional neural network that is trained to output 1 given an image of a car and otherwise 0. A generative model on the other hand doesn’t know anything about classes of data. Instead, its purpose is to generate new data which fits the distribution of the training data.

2.2 Basic Architecture

GANs consist of two main components which are the Generator and the Discriminator. Think it like a game where Generator tries to produce some data from probability distribution and Discriminator acts like a judge. Basically,

Generator generates fake data which are really close to the realistic data in order to fool the discriminator. While the discriminator tries to learn by distinguishing which data is fake and which data is real.

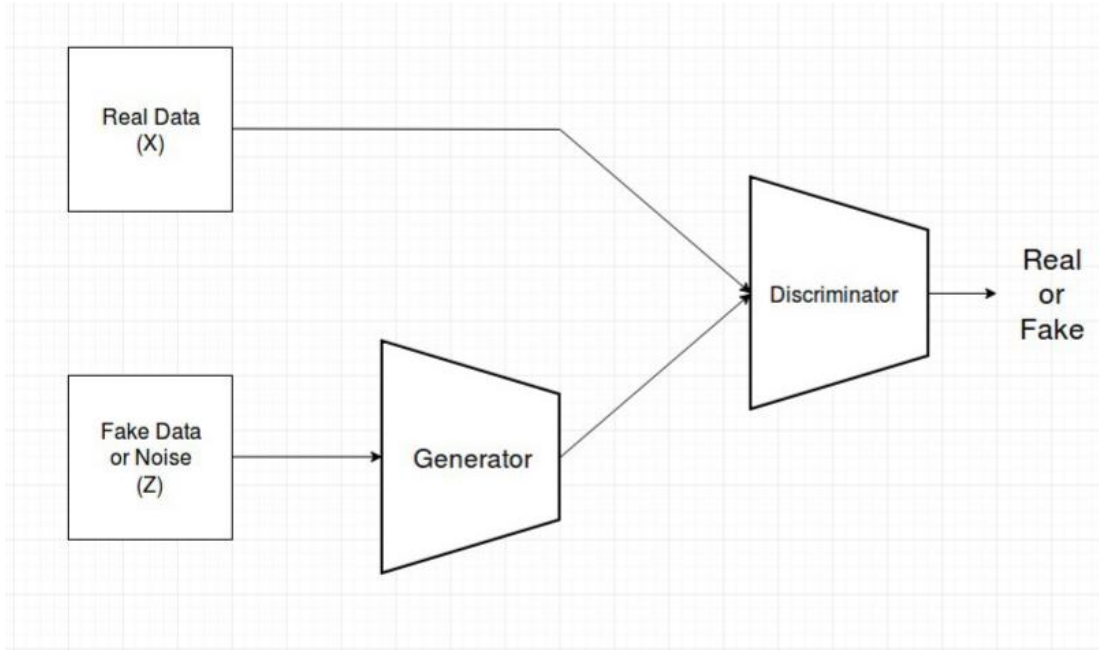


Fig 2. GAN basic principle

2.2.1 Adversarial Network Architecture

The generator's architecture is shown in Figure 3, which is a modification from SRResNet. The model contains 16 ResBlocks and uses 3 sub-pixel CNN for feature map upscaling. Figure 4 shows the discriminator architecture, which contains 10 Resblocks in total. All batch normalization layers are removed in the discriminator, since it would bring correlations within the mini-batch, which is undesired for the computation of the gradient norm. We add an extra fully-connected layer to the last convolution layer as the attribute classifier. All

weights are initialized from a Gaussian distribution with mean 0 and standard deviation 0.02.

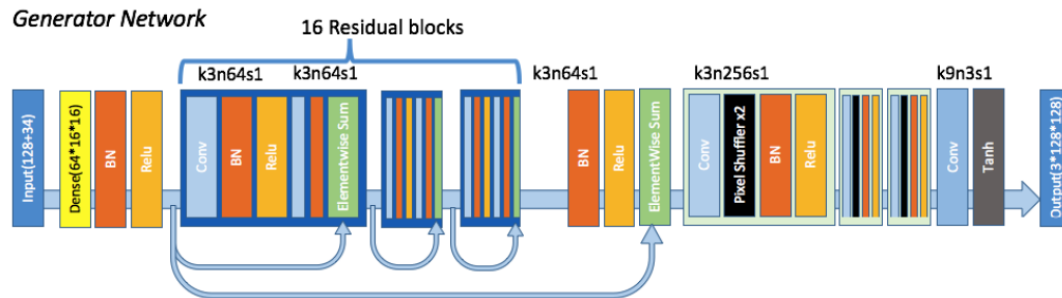


Fig 3. Generator Model Architecture

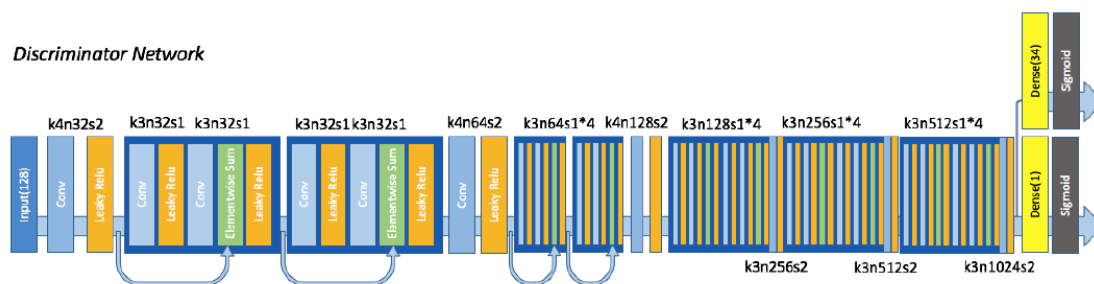


Fig 4. Discriminator Model Architecture

2.3 Components of GAN

The components of GAN, Generator and Discriminator, are CNNs (Convolutional Neural Networks) as they process the image.

2.3.1 Convolutional Neural Networks

A Convolutional neural network (CNN) is a neural network that has one or more convolutional layers and are used mainly for image processing, classification, segmentation and also for other auto correlated data. A convolution is essentially sliding a filter over the input.

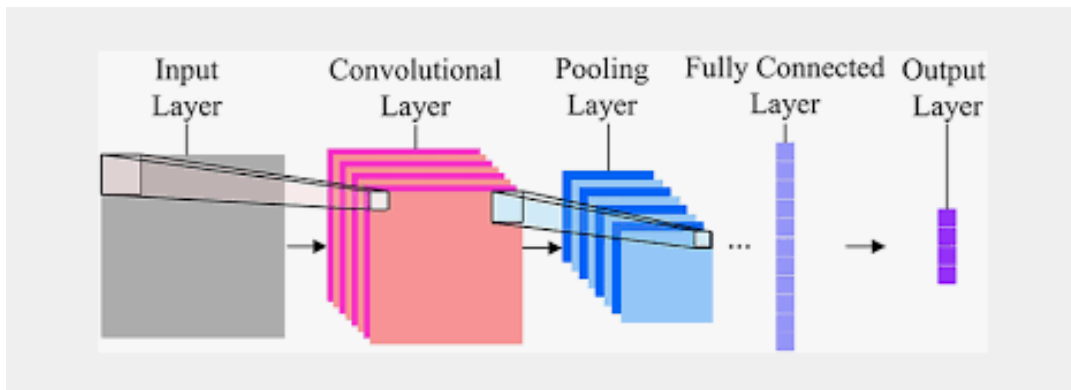


Fig 5. CNN Components

The state of the art for many computer vision problems is meanwhile set by specifically designed CNN architectures. It was shown that deeper network architectures can be difficult to train but have the potential to substantially increase the network's accuracy as they allow modelling mappings of very high complexity.

To efficiently train these deeper network architectures, batch normalization is often used to counteract the internal co-variate shift. Deeper network architectures have also been shown to increase performance for image generation.

2.3.2 Challenges of a CNN

The main challenges for a neural network are learning speed and training set size. In order to generalize well, a neural network must have enough training data to learn from. It also needs to be able to learn quickly enough so that it can adapt to changing conditions as it is deployed.

The other main challenge is being able to provide the network with sufficient computing power to train itself. A modern deep neural network is a massive parallel computation, and as such requires a large amount of processing power to train.

The major challenges of a neural network and deep neural network are:

1. Data hungry: the models tend to overfit if you don't have a lot of data to train them on.
2. Compute hungry: they tend to be very computationally intensive and as such mere mortals like me with just a laptop cannot do much, or at least we cannot try out complex state of the art models like the researchers and more wealthy can. Runtime has the same problem though not as pronounced.
3. Training time: even on multiple GPUs some models take a long time to train.
4. Interpretability: they have millions, or even billions of parameters with orders of magnitude more relationships between said parameters. We therefore cannot know what relationships or parameters led to a conclusion, or why exactly it came to such a conclusion.

2.3.3 Loss Functions

This is the most important part. As discussed we will be using Perceptual loss. It comprises of Content (Reconstruction) loss and Adversarial loss.

$$l^{SR} = \underbrace{l_X^{SR}}_{\text{content loss}} + \underbrace{10^{-3}l_{Gen}^{SR}}_{\text{adversarial loss}}$$

perceptual loss (for VGG based content losses)

Adversarial loss: This pushes our solution to the natural image manifold using a discriminator network that is trained to differentiate between the super-resolved images and original photo-realistic images.

$$l_{Gen}^{SR} = \sum_{n=1}^N -\log D_{\theta_D}(G_{\theta_G}(I^{LR}))$$

Content Loss: Content loss we are using so that we can keep perceptual similarity instead of pixel wise similarity. This will allow us to recover photo-realistic textures from heavily down sampled images. Instead of relying on pixel-wise losses we will use a loss function that is closer to perceptual similarity. We define the VGG loss based on the ReLU activation layers of the pre-trained 19 layer VGG network. VGG loss is defined as the euclidean distance between the feature representations of a reconstructed image and the reference image.

$$l_{VGG/i,j}^{SR} = \frac{1}{W_{i,j}H_{i,j}} \sum_{x=1}^{W_{i,j}} \sum_{y=1}^{H_{i,j}} (\phi_{i,j}(I^{HR})_{x,y} - \phi_{i,j}(G_{\theta_G}(I^{LR}))_{x,y})^2$$

Pixel-wise loss functions such as MSE (Mean Square Error) struggle to handle the uncertainty inherent in recovering lost high-frequency details such as texture: minimizing MSE encourages finding pixel-wise averages of plausible solutions which are typically overly-smooth and thus have poor perceptual quality. Reconstructions of varying perceptual quality the problem of minimizing MSE where multiple potential solutions with high texture details are averaged to create a smooth reconstruction.

MSE (Mean Square Error) is the loss/cost function most ordinarily used regression loss function. MSE is that the sum of squared distances between our target variable and predicted values Mean squared error may be a loss function used for regression once you don't need outliers to play an enormous role. Used as a performance metric, it is easy to interpret.

2.3.4 Activation Function

Activation functions take any real number as input, also known as its domain, and outputs a number in a certain range using a non-linear differentiable function. We typically use them for classification between certain layers in deep neural networks and more specifically in games.

Activation functions should be differentiable and non-linear because it is used for backpropagation for training neural network and updating its parameters. It needs to be able to differentiate and provide gradients to the previous layer. It also needs to be non-linear. The features that are computed within the neural networks can be complex. If we didn't use non-linear activations, a neural

network like this one with multiple hidden layers and neurons could actually be collapsed into a simple linear regression.

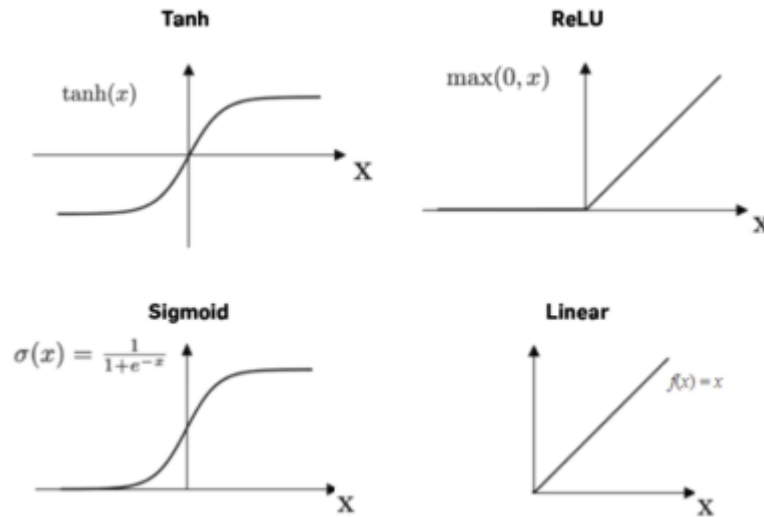


Fig 6. Activation Functions

2.3.5 Backpropagation

Backpropagation is a very common technique for preparing artificial neural organizations and it is based on an inclination based improvement strategy, for example, stochastic angle plunge. In backpropagation, we initially figure the forecast and its related loss for each preparation model. At that point we proliferate the mistake to register the fractional subsidiaries of the cost work C for all loads w and predispositions b in the fake neural organizations.

CHAPTER 3

METHODOLOGY

3.1 Our Approach

Generative Adversarial Networks consist of two neural networks: a discriminator and a generator. The discriminator receives both real images from the training set and generated images produced by the generator. The discriminator outputs the probability that an image is real, so it is trained to output high values for the real images and low values for the generated ones. The generator is trained to produce images that the discriminator thinks are real. Both the discriminator and generator are trained simultaneously so that they compete against each other. As a result of this, the generator learns to produce more and more realistic images as it trains.

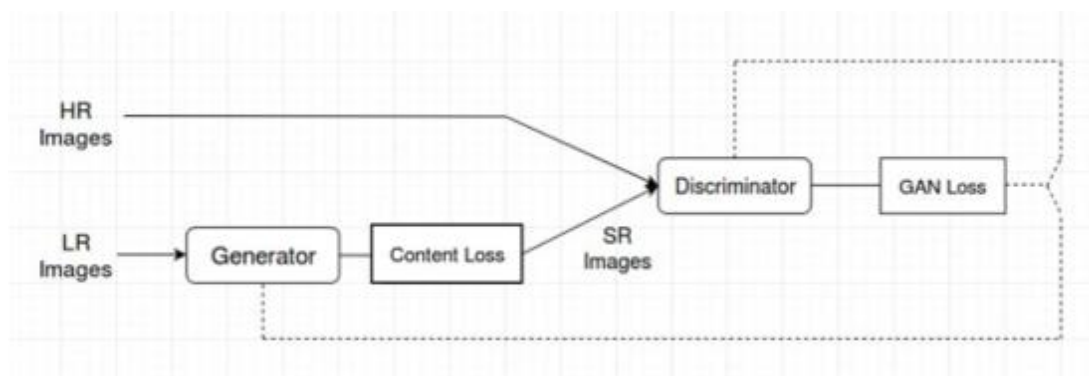


Fig 7. Our GAN's approach

Our ultimate goal is to train a generating function G that estimates for a given set of input tags a corresponding image. To achieve this, we train a generator

network as a feed-forward CNN with residual blocks, and we also train a discriminator network as a fully connected CNN used as a classifier.

3.2 Design of the Model

The generator's architecture is shown below, which is a modification from SRResNet. The model contains 16 ResBlocks and uses 3 sub-pixel CNN for feature map upscaling. The discriminator architecture, which contains 10 Resblocks in total. All batch normalization layers are removed in the discriminator, since it would bring correlations within the mini-batch, which is undesired for the computation of the gradient norm. We add an extra fully-connected layer to the last convolution layer as the attribute classifier. All weights are initialized from a Gaussian distribution with mean 0 and standard deviation 0.02.

Training Model :

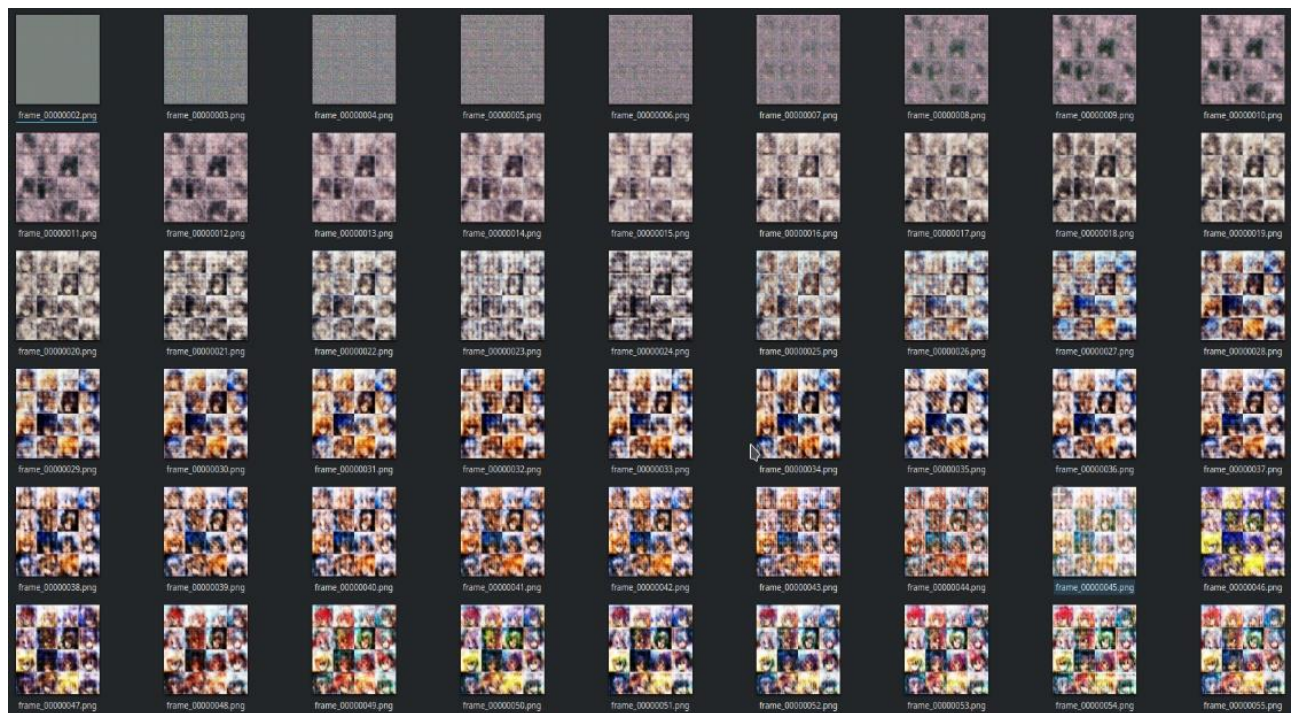


Fig 8. Images during training (1 picture per 20 epochs)

Generator Model:

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	(None, 1, 1, 100)	0
conv2d_transpose_1 (Conv2DTr	(None, 4, 4, 512)	819712
batch_normalization_4 (Batch	(None, 4, 4, 512)	2048
leaky_re_lu_5 (LeakyReLU)	(None, 4, 4, 512)	0
conv2d_transpose_2 (Conv2DTr	(None, 8, 8, 256)	2097408
batch_normalization_5 (Batch	(None, 8, 8, 256)	1024
leaky_re_lu_6 (LeakyReLU)	(None, 8, 8, 256)	0
conv2d_transpose_3 (Conv2DTr	(None, 16, 16, 128)	524416
batch_normalization_6 (Batch	(None, 16, 16, 128)	512
leaky_re_lu_7 (LeakyReLU)	(None, 16, 16, 128)	0
conv2d_transpose_4 (Conv2DTr	(None, 32, 32, 64)	131136
batch_normalization_7 (Batch	(None, 32, 32, 64)	256
leaky_re_lu_8 (LeakyReLU)	(None, 32, 32, 64)	0
conv2d_6 (Conv2D)	(None, 32, 32, 64)	36928
batch_normalization_8 (Batch	(None, 32, 32, 64)	256
leaky_re_lu_9 (LeakyReLU)	(None, 32, 32, 64)	0
conv2d_transpose_5 (Conv2DTr	(None, 64, 64, 3)	3075
Total params: 3,616,771		
Trainable params: 3,614,723		
Non-trainable params: 2,048		

Discriminator model:

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	(None, 64, 64, 3)	0
conv2d_1 (Conv2D)	(None, 32, 32, 64)	3136
leaky_re_lu_1 (LeakyReLU)	(None, 32, 32, 64)	0
conv2d_2 (Conv2D)	(None, 16, 16, 128)	131200
batch_normalization_1 (Batch Normalization)	(None, 16, 16, 128)	512
leaky_re_lu_2 (LeakyReLU)	(None, 16, 16, 128)	0
conv2d_3 (Conv2D)	(None, 8, 8, 256)	524544
batch_normalization_2 (Batch Normalization)	(None, 8, 8, 256)	1024
leaky_re_lu_3 (LeakyReLU)	(None, 8, 8, 256)	0
conv2d_4 (Conv2D)	(None, 4, 4, 512)	2097664
batch_normalization_3 (Batch Normalization)	(None, 4, 4, 512)	2048
leaky_re_lu_4 (LeakyReLU)	(None, 4, 4, 512)	0
conv2d_5 (Conv2D)	(None, 1, 1, 100)	819300
Total params: 3,579,428		
Trainable params: 3,577,636		
Non-trainable params: 1,792		

3.3 Dats Set and Experimental Tools Setup

anime-faces Dataset

Anime-style images of 126 tags are collected from danbooru.donmai.us using the crawler tool gallery-dl. The images are then processed by a anime face detector python-animeface. The resulting dataset contains ~143,000 anime faces.

Experimental Tools used

1. TensorFlow
2. Keras
3. Jupyter Notebook
4. Spyder

Data Overview

It is well understood that image dataset in high quality is essential, if not most important, to the success of image generation. Web services hosting images such as Danbooru² and Safebooru³, commonly known as image boards, provide access to a large number of images enough for training image generation models. Previous works mentioned above all base their approaches on images crawled from one of these image boards, but their datasets suffer from high inter-image variance and noise. We hypothesize that it is due to the fact that image boards allow uploading of images highly different in style, domain, and quality, and believe that it is responsible for a non-trivial portion

of quality gaps between the generation of real people faces and anime character faces.

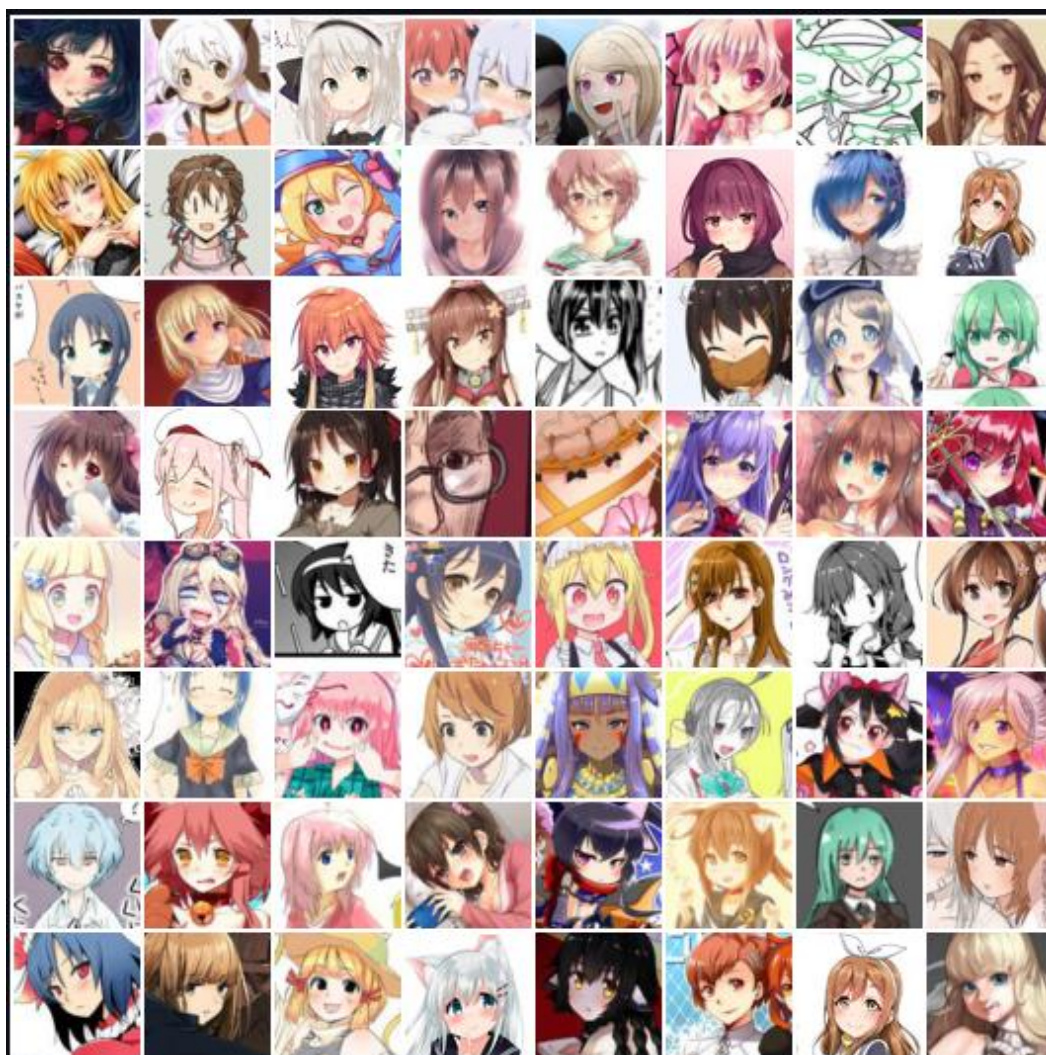


Fig 9. Sample training images.

CHAPTER 4

RESULTS AND APPLICATIONS

4.1 Results

Therefore automatic generation of facial images has been achieved.



Fig 10. Results after 5000 epochs

4.2 Applications

We all love anime characters and are tempted to create our custom ones. However, it takes tremendous efforts to master the skill of drawing, after which we are first capable of designing our own characters. To bridge this gap, the automatic generation of anime characters offers an opportunity to bring a custom character into existence without professional skill. Besides the benefits for a non-specialist, a professional creator may take advantages of the automatic generation for inspiration on animation and game character design; a RPG developer may employ copyright-free facial images to reduce designing costs in game production.

4.3 Qualitative Analysis

Attribute Precision

To evaluate how each tag affects the output result, we measure the precision of the output result when the certain label is assigned. With each target, we fix the target label to true, and sample other labels in random. For each label, 20 images are drawn from the generator. Then we manually check generated results and judge whether output images behave the fixed attribute we assigned.

FID Evaluation

One possible quantitative evaluation method for GAN model is Fréchet Inception Distance(FID). To calculate the FID, they use a pre-trained CNN(Inception model) to extract vision-relevant features from both real and fake samples. The real feature distribution and the fake feature distribution are approximated with two gaussian distributions. Then, they calculate The Fréchet

distance(Wasserstein-2 distance) between two gaussian distributions and serve the results as a measurement of the model quality. The Inception model trained on ImageNet is not suitable for extracting features of anime-style illustrations, since there is no such images in the original training dataset. Here, we replace the model with Illustration2vec feature extractor model for better measurement of visual similarities between generated images and real images.

To evaluate the FID score for our model, we sample 12800 images from real dataset, then generate a fake sample by using the corresponding conditions for each samples real images. After that we feed all images to the Illustration2vec feature extractor and get a 4096-dimension feature vector for each image. FID is calculated between the collection of feature vectors from real samples and that from fake samples.

REFERENCES

1. C. Dong, C. C. Loy, K. He, and X. Tang. Image super-resolution using deep convolutional networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 38(2):295–307, 2016.
2. J. Johnson, A. Alahi, and F. Li. Perceptual losses for real-time style transfer and super- resolution. In *European Conference on Computer Vision (ECCV)*, pages 694–711. Springer, 2016
3. W. Shi, J. Caballero, F. Huszar, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang. Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1874–1883, 2016.
4. Yanghua Jin, Jiakai Zhang, Towards the Automatic Anime Characters Creation with Generative Adversarial Networks arXiv:1708.05509v1
5. <https://github.com/tdrussell/IllustrationGAN>
6. <https://github.com/pavitrakumar78/Anime-Face-GAN-Keras>
7. <https://github.com/jayleicn/animeGAN>
8. <https://github.com/forcecore/Keras-GAN-Animeface-Character>

Modelling Illustrations using Generative Adversarial Networks

ORIGINALITY REPORT

16%	14%	7%	14%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Netaji Subhas Institute of Technology Student Paper	4%
2	mc.ai Internet Source	3%
3	data.vision.ee.ethz.ch Internet Source	3%
4	www.repository.cam.ac.uk Internet Source	2%
5	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	1%
6	ohmisskate.com Internet Source	1%
7	Submitted to Tezpur University - CN-173457 Student Paper	1%
8	www.technologynetworks.com Internet Source	<1%

9	Submitted to essex Student Paper	<1 %
10	priyanshi20.files.wordpress.com Internet Source	<1 %
11	www.ijetjournal.org Internet Source	<1 %
12	Marszal-Pomianowska, Anna, Per Heiselberg, and Olena Kalyanova Larsen. "Household electricity demand profiles – A high-resolution load model to facilitate modelling of energy flexible buildings", Energy, 2016. Publication	<1 %
13	export.arxiv.org Internet Source	<1 %

Exclude quotes On
Exclude bibliography On

Exclude matches Off